

RELATÓRIO TÉCNICO DE GOVERNANÇA E ÉTICA

TaskAnalyzer · Bootcamp III · Entrega Final (Fase 3 de 3)

Identificação

Nome completo	Juan Conceição Cassimiro
Curso	Ciências da Computação
Polo/Turma	Turma A - 0726
E-mail institucional	juan.cassimiro67@sempreceub.com
Atividade	Desafio Final — DevOps, Testes Automatizados, Containerização e Governança Ética de IA
Data	18/10/2026

Links da Entrega

Repositório GitHub (público)
https://github.com/juan1cassimiroo1/sdd-task-analyzer

Pitch técnico em vídeo (YouTube, 5 minutos)
[inserir o link do YouTube após publicar o vídeo]

Código de Ética da EaD: declaro que mantenho os princípios de autoria individual na elaboração deste trabalho e da gravação do vídeo. O uso de assistente de IA está declarado e rastreado na seção 5.3.

1. Objetivo e Escopo

Este relatório documenta a Fase 3 (Entrega Final) do projeto TaskAnalyzer: a esteira de integração contínua, a containerização, a governança do código gerado por IA e a homologação humana final. Ele demonstra como a especificação (SDD) e as regras de contexto (CONTEXT_RULES.md) delimitaram o trabalho do assistente de IA e quais falhas a revisão humana ainda encontrou.

2. Arquitetura Final do Repositório

```
sdd-task-analyzer/
├── .github/workflows/ci.yml      # Motor CI/CD (GitHub Actions)
├── Dockerfile                    # Empacotamento (python:3.11-slim)
├── docker-compose.yml            # Orquestração da execução isolada
├── CONTEXT_RULES.md              # Base de governança (Fase 1)
├── specs/
│   ├── system_design_document.pdf  # Documento SDD entregue na Fase 1
│   ├── task_analyzer_spec.md       # Especificação em Markdown (v1.1)
│   └── requirements.yaml           # Contrato legível por máquina
├── tests/test_harness.py         # Suíte de testes (Fase 2)
├── src/task_analyzer.py          # Código gerado por IA e homologado
├── docs/                         # Este relatório
├── requirements.txt
└── README.md
```

3. Pilar 1 — Integração Contínua (GitHub Actions)

O workflow .github/workflows/ci.yml é disparado por push e por Pull Request nas branches main e develop. O job de testes roda em ubuntu-latest com Python 3.11, instala as dependências de requirements.txt, verifica o estilo (flake8) e executa pytest tests/test_harness.py -v. Um segundo job, que depende do primeiro, constrói a imagem Docker e executa a suíte dentro do container.

Etapa do pipeline	Comando	Falha bloqueia?
Ambiente	ubuntu-latest · Python 3.11	—
Dependências	pip install -r requirements.txt	Sim
Estilo (PEP 8)	flake8 src tests --max-line-length=100	Sim
Test Harness	pytest tests/test_harness.py -v	Sim
Imagem Docker	docker build · docker run · docker compose run	Sim

Evidência: a aba Actions do repositório (https://github.com/juan1cassimiroo1/sdd-task-analyzer/actions) lista as execuções. As execuções da versão v0.3.0 (main, develop e feature/cicd-docker-fase3) terminaram com sucesso; o workflow desta entrega foi alinhado ao enunciado e é reexecutado a cada push.

4. Pilar 2 — Containerização (Docker)

O Dockerfile parte de python:3.11-slim (ambiente leve e imutável), instala as dependências, copia o código e define o pytest como comando padrão (CMD). O container executa como usuário não-root. O docker-compose.yml define o serviço task-analyzer e permite disparar a suíte inteira, isolada, com um único comando.

```
docker build -t sdd-task-analyzer .
docker run --rm sdd-task-analyzer
docker compose run --rm task-analyzer
```

	Execução local	Ambiente Docker
Dependências	Configuração manual do sistema	Imagem isolada e imutável (python:3.11-slim)
Consistência	Sujeita a variáveis ocultas do ambiente	Mesmo resultado em qualquer máquina
Execução	Vários comandos e instalações prévias	Um único comando padronizado

5. Pilar 3 — Governança e Ética de IA

5.1 Análise de Alucinações e Desvios

O assistente de IA recebeu como contexto a especificação (SDD) e o CONTEXT_RULES.md. A tabela separa o que essas barreiras bloquearam por construção do que ainda foi encontrado na revisão humana — esses últimos são os desvios reais do código gerado.

#	Situação	Classificação	Tratamento e evidência
1	Tarefa concluída sem data_inicio: o código usava data_criacao como início.	Desvio real da IA (lógica fora do contrato)	Contrariava a regra "não assumir comportamentos não especificados". Passa a lançar DataInvalidaError. Teste: test_tarefa_concluida_sem_data_inicio.
2	logging.basicConfig executado na importação do módulo.	Desvio de boa prática	Uma biblioteca não deve alterar o logging de quem a importa. Removido.
3	Exceções de validação sem classe-base comum.	Lacuna de design	Criada TaskValidationError; as três exceções do contrato herdam dela. Testes de herança adicionados.
4	Bibliotecas não autorizadas.	Risco bloqueado	src/task_analyzer.py importa apenas a biblioteca padrão (logging, dataclasses, datetime, enum, typing). requirements.txt só tem pytest (desenvolvimento).
5	Divisão por zero sem tarefas concluídas.	Risco bloqueado pelo contrato	NenhumaTarefaConcluidaError é lançada antes de qualquer divisão.
6	Persistência de dados e mudança de assinatura pública.	Risco bloqueado	Nenhuma operação de arquivo ou banco. analisar_tarefas(tarefas) -> ResultadoAnalise permanece inalterada.
7	Alteração dos cenários de teste fornecidos.	Risco bloqueado	Os 10 testes originais estão intactos; os casos novos ficam na classe TestHomologacaoFase3.

5.2 Homologação Humana — Modelo Critical-in-the-Loop

Etapa	O que acontece	Aplicação neste projeto
1. Geração	O agente de IA implementa a lógica central de análise de tarefas.	src/task_analyzer.py gerado com o SDD e o CONTEXT_RULES como contexto.
2. Barreira SDD	O código passa pelo contrato executável e pelas regras de contexto.	Conferência contra specs/task_analyzer_spec.md e requirements.yaml.
3. Validação humana	O engenheiro QA procura inconsistências: divisão por zero, datas inválidas (TaskValidationError).	Desvios 1 a 3 da tabela acima foram encontrados nesta etapa.
4. Integração segura	Só o código homologado entra em src/task_analyzer.py.	Merge condicionado ao pipeline verde.

Decisões que ficaram com o humano, e não com a IA:

- Prioridade sem tarefas concluídas devolve métricas zeradas (e não exceção), por ser uma segmentação normal dos dados — registrado na seção 4 da especificação.
- Tarefa concluída exige data_inicio e data_conclusao: o contrato não autoriza estimar uma data ausente.
- Tarefas canceladas ou pendentes contam em quantidade_tarefas, mas não entram nas métricas de tempo e atraso.

5.3 Sustentabilidade, Rastreabilidade e Ética

Assistente de IA utilizado: Claude (Anthropic), em apoio à redação da especificação, à geração do código, da esteira e deste relatório. A autoria das decisões de design, a revisão crítica e a aprovação final são do autor. Cada etapa deixa um rastro verificável:

Fase	Contexto entregue à IA	Artefato gerado	Verificação humana
1	Enunciado do desafio (SDD e AI Harness)	Especificação SDD, CONTEXT_RULES	Conferência com o barema; publicação no Google Docs e vídeo

Fase	Contexto entregue à IA	Artefato gerado	Verificação humana
2	Especificação, CONTEXT_RULES e restrições	src/task_analyzer.py, tests/test_harness.py	Testes, flake8, fluxo feature → develop → main, tag v0.2.0
3	Enunciado da Fase 3 e repositório existente	ci.yml, Dockerfile, docker-compose.yml, correções, relatório	Revisão do código, 15 testes, CI no GitHub

- Rastreabilidade técnica: histórico de commits, branches e tags no GitHub; logs de cada execução do CI.
- Privacidade (LGPD): o módulo não persiste nem transmite dados e não faz chamadas de rede. O campo titulo é texto livre e não é anonimizado — em uso real, não deve conter dados pessoais.
- Equidade: as três prioridades são tratadas de forma simétrica, sem pesos privilegiados.
- Limitação conhecida: todas as datas devem chegar em UTC; não há conversão de fuso horário.
- Sustentabilidade: imagem slim, usuário não-root, sem estado persistente e sem dependências de execução além da biblioteca padrão.

6. Evidências de Verificação

Verificação	Resultado
Suíte pytest (tests/test_harness.py)	15 de 15 testes aprovados (10 originais + 5 de homologação)
flake8 (PEP 8)	Sem apontamentos
Sintaxe do workflow e do docker-compose.yml	YAML válido
Execuções anteriores do CI (v0.3.0)	3 de 3 com sucesso: main, develop e feature/cicd-docker-fase3
Docker	Build e execução validados pelo job docker do CI e demonstrados no vídeo

7. Conclusão

O TaskAnalyzer entrega confiabilidade técnica (Docker e CI/CD: portável e automatizado) e confiabilidade ética (SDD e governança: alinhado e controlado). O caso do início da tarefa mostra por que a homologação humana permanece indispensável: mesmo com contrato e regras de contexto, a IA preencheu uma lacuna com uma suposição, e foi a revisão crítica, amparada por um teste, que a corrigiu.